

遗传因子库计算明细

本文对应当前正式文件 `outputs/factor_library.json` 中的 20 个因子。这里列的是实际运行公式，不是重新整理后的人工版本。

计算顺序

1. `feature_builder.py` 从四张基础表构造日级基础特征。
2. 缺失的横截面排名填为 `0.5`，其他连续特征填为 `0.0`。
3. 遗传表达式逐行计算，结果限制在 `[-1e6, 1e6]`。
4. 遗传表达式中的除法在分母绝对值不大于 `1e-8` 时返回缺失值。
5. 决策树和 LightGBM 使用前，会再次把每个遗传因子的公式结果转换成当日横截面分位数。
6. IC 投票策略也先做当日横截面排名，再根据近期 IC 决定正向或反向。

因此，这些公式的主要意义是定义股票之间的排序形状。公式混合了排名、比例和技术指标，不应把最终数值解释为带有经济单位的量。

遗传算子

令输入为 `x`、`a`、`b`：

```
neg(x)          = -x
abs(x)          = |x|
signed_sqrt(x)  = sign(x) * sqrt(|x|)
signed_log(x)   = sign(x) * log(1 + |x|)
add(a, b)       = a + b
sub(a, b)       = a - b
mul(a, b)       = a * b
div(a, b)       = a / b, 要求 |b| > 1e-8
min(a, b)       = 逐样本较小值
max(a, b)       = 逐样本较大值
```

遗传算法完整实现

这一节描述当前代码实际执行的遗传搜索，不是一般意义上的遗传编程概念说明。主要实现位于 `factor_engine.py` 和 `build_factor_library.py`，正式参数来自 `config.json`。

1. 搜索数据准备

遗传搜索不会在全部 1,100 多万行数据上反复计算每个表达式，而是先建立一份固定搜索样本：

1. 只保留 2025-12-31 及以前、并且已经具有未来 5 日收益标签的数据。
2. 从全部可用历史月份中，使用 NumPy 随机数生成器和固定种子 42，无放回抽取 48 个月。
3. 对每个交易日的股票代码计算固定的 BLAKE2 哈希值，按哈希值排序后取前 700 只股票。
4. 按交易日、股票代码重新排序，预先计算每日未来 5 日收益的横截面分位数排名。

本次正式搜索最终使用：

历史月份数	48
每日股票上限	700
搜索样本行数	688,800
随机种子	42

BLAKE2 哈希抽样的作用是保证同一股票在不同进程中得到相同的抽样优先级，避免 Python 内置哈希随机化导致每次搜索样本变化。这里的 48 个月是随机分散在 2026 年以前的历史中，并不是连续 48 个月。

2. 染色体如何表示

一个遗传个体就是一棵表达式语法树，代码中使用嵌套列表保存：

```
终端节点: "turnover_rate_f"  
一元节点: [算子, 子表达式]  
二元节点: [算子, 左表达式, 右表达式]
```

当前元数据共有 67 个基础特征，其中以下 7 个缺失或状态字段被排除：

```
is_basic_missing  
is_moneyflow_missing  
is_tech_warmup  
is_recent_listing  
valid_feature_ratio  
valuation_missing_count  
is_loss_or_pe_missing
```

因此遗传搜索共有 60 个可选终端特征。

例如 factor_01 的实际染色体是：

```
[
  "mul",
  "turnover_rate_f",
  [
    "add",
    "macd_dea",
    ["signed_sqrt", "turnover_rank_cs"]
  ]
]
```

对应公式：

```
turnover_rate_f * (macd_dea + signed_sqrt(turnover_rank_cs))
```

它的根节点 `mul` 为第 1 层，`add` 为第 2 层，`signed_sqrt` 为第 3 层，所以表达式深度为 3，总节点数为 6。

表达式会被序列化为无多余空格的 JSON 字符串，作为缓存键。这个规范化只识别语法树完全相同的表达式：

```
add(a, b) 和 add(b, a) 会被视为两个不同个体
neg(neg(a)) 和 a 也会被视为两个不同个体
```

系统目前没有代数化简或等价表达式识别。

3. 初始种群如何生成

正式搜索的初始种群包含 80 个随机表达式，最大允许深度为 3。

随机生成一棵表达式时：

1. 根节点深度为 0，不能直接生成基础特征，因此根节点必定是算子。
2. 生成算子节点时，有 35% 概率选择一元算子，有 65% 概率选择二元算子。
3. 到达非根节点后，每个位置有 30% 概率提前停止并随机选择一个基础特征。
4. 到达深度 3 时强制停止，随机选择基础特征。

因此初始个体至少包含一个算子，但不一定都达到深度 3。随机移民也使用完全相同的生成方法。

4. 适应度如何计算

每个表达式先在 688,800 行搜索样本上逐行计算。计算结果会裁剪到 $[-1e6, 1e6]$ 。如果表达式完全没有有效值，或者有效值标准差低于 $1e-6$ ，其适应度直接设为 $-1e9$ ，相当于淘汰。

对有效表达式，适应度按以下顺序计算：

1. 每个交易日把表达式值转换成股票横截面分位数排名。
2. 计算表达式排名与未来 5 日收益排名之间的逐日 Pearson 相关；这等价于横截面 Spearman Rank IC。
3. 每日至少需要 200 只有效股票，否则该日 IC 不参与统计。
4. 按 YYYYMM 汇总逐日 IC，得到每个月的平均 IC。
5. 每个月至少需要 12 个有效交易日，否则该月不参与适应度。
6. 对所有有效月份的月均 IC 取绝对值，选择最大的 3 个月。

最终分数为：

```
fitness
= mean(top3(abs(monthly_mean_ic)))
  - 0.05 * std(daily_ic)
  - 0.0005 * expression_nodes
```

三部分分别表示：

阶段性能力奖励	历史表现最强的 3 个月绝对 IC 均值
日度噪声惩罚	逐日 IC 标准差的 5%
复杂度惩罚	每个语法树节点扣 0.0005

适应度使用绝对月 IC，因此正向和反向因子地位相同。它不要求因子长期方向一致，只要求至少部分历史月份具有较强横截面排序能力。

`peak_month` 是上述 Top-3 月份中绝对 IC 最大的月份，后续用于限制因子库过度集中在同一个市场阶段。

5. 一代遗传的完整过程

正式搜索执行 20 代。每一代的操作顺序固定如下：

1. 计算当前 80 个个体的适应度。
2. 按适应度从高到低排序。
3. 将前 12 个表达式作为精英，原样复制到下一代。
4. 从这 12 个精英中均匀随机选择父本，生成 56 个后代，使下一代先达到 68 个个体。
5. 再随机生成 12 个全新表达式作为随机移民。
6. 合并为下一代的 80 个个体。

数量关系为：

```
12 个精英
+ 56 个交叉/变异后代
+ 12 个随机移民
= 80 个下一代个体
```

对每个后代，交叉和变异依次、独立判断：

交叉概率 = 55%

变异概率 = 35%

所以一个后代可能出现以下四种情况：

不交叉、不变异

只交叉

只变异

先交叉，再对交叉结果变异

精英本身不会被交叉或变异。随机移民也不会在加入当代时再次变异。

6. 交叉操作的真实行为

交叉以精英池中均匀随机抽取的两个表达式作为父本。父本一提供主体结构，父本二提供被插入的整棵表达式。

递归交叉过程是：

1. 从父本一当前节点开始。
2. 如果当前节点已经是基础特征，直接用父本二整棵表达式替换它。
3. 如果当前节点是算子，则有 30% 概率在当前位置停止，并用父本二整棵表达式替换当前子树。
4. 如果没有停止，就在当前节点的一个子分支中均匀随机选择一支，继续向下递归。
5. 未被选中的父本一分支保持不变。

例如：

父本一: `mul(a, add(b, c))`

父本二: `signed_sqrt(d)`

一种可能后代: `mul(a, add(signed_sqrt(d), c))`

这不是“两个父本各随机切一棵子树再互换”的对称交叉，而是把父本二的完整表达式插入父本一的某个位置。代码通过深拷贝保证交叉不会原地修改父本。

7. 变异操作的真实行为

变异处理交叉后的表达式；如果该后代没有发生交叉，则直接处理父本一。

递归变异过程是：

1. 如果当前位置已经是基础特征，直接用一棵新随机子树替换它。

2. 如果当前位置是算子，则有 25% 概率在当前位置停止，用新随机子树替换整个当前子树。
3. 如果没有停止，就在当前节点的一个子分支中均匀随机选择一支，继续向下递归。
4. 递归越深，允许新生成子树使用的深度预算越小。

例如：

```
变异前: mul(a, add(b, c))
变异后: mul(a, add(b, signed_log(d)))
```

交叉和变异都可能临时生成超过全局最大深度 3 的表达式。每个后代完成全部操作后会重新计算实际深度：

```
如果 depth <= 3: 保留后代
如果 depth > 3: 丢弃整个后代，重新随机生成一棵深度不超过 3 的表达式
```

当前实现不会只裁掉超深分支，而是直接替换整个违规后代。

8. 适应度缓存与历史池

每棵语法树先通过 JSON 规范化得到唯一字符串键。

```
cache: 保存该语法树已经计算过的 Fitness 和逐日 IC
hall: 保存所有代中出现过的唯一表达式及其最好记录
```

如果同一语法树在精英保留、交叉或随机生成中再次出现，系统直接读取缓存，不重复计算适应度。hall 不只保存最后一代，而是持续收集全部 20 代出现过的表达式。

本次正式搜索最终情况：

总代数	20
每代种群	80
历史唯一表达式	918

918 小于理论上的 $80 * 20 = 1600$ ，主要因为精英会重复出现，部分后代也可能与历史表达式完全相同。

9. 最终 20 个因子如何选出

20 代结束后，不是直接取最后一代前 20 名，而是在全部历史池中筛选：

1. 按搜索适应度从高到低排序。
2. 最多取前 600 个作为候选。
3. 按候选顺序逐个检查，使用贪心方式决定是否加入因子库。
4. 同一个搜索峰值月份最多允许保留 3 个因子。
5. 把候选公式值按交易日转为横截面排名。

6. 与已经保留的每个因子计算搜索样本上所有排名值的 Pearson 相关。
7. 如果任一绝对相关系数达到或超过 0.80，认为候选与已有因子重复，跳过。
8. 直到保留满 20 个因子，或者候选耗尽。

这里的相关性去重使用拼接后的全部搜索样本排名值，不是先计算每日因子相关系数再取平均。

完整历史验证由 `validate_factor_library.py` 在因子库建立后执行。它会用全部 2026 年前股票和日期重新计算月 IC，并写入：

```
full_peak_month  
full_best_month_abs_ic  
full_top3_month_abs_ic  
full_mean_abs_month_ic  
full_months_abs_ic_ge_005
```

全量验证只补充统计信息，不会根据验证结果重新排序、替换或删除已经选中的 20 个因子。因此搜索样本上的 `peak_month` 与全量验证后的 `full_peak_month` 可能不同。

10. 完整伪代码

读取 2026 年前数据和未来 5 日收益标签

随机抽取 48 个历史月份

每天按 BLAKE2 股票哈希固定抽取 700 只股票

预计算每日标签横截面排名

population = 随机生成 80 棵深度不超过 3 的表达式树

cache = 空适应度缓存

hall = 空历史表达式池

重复 20 代:

 scored = []

 对 population 中每个 expr:

 key = expr 的规范化 JSON

 如果 key 不在 cache:

 计算 expr 的逐日横截面 Rank IC

 计算有效月均 IC

 fitness = Top-3 绝对月 IC 均值

 - 日 IC 波动惩罚

 - 节点数惩罚

 写入 cache

 将 expr 和 fitness 加入 scored

 将 expr 写入 hall

 scored 按 fitness 降序排列

 elites = 前 12 个 expr

 next_population = elites

 当 next_population 数量小于 68:

 parent_1 = 从 elites 均匀随机抽取

 child = parent_1

 以 55% 概率:

 parent_2 = 从 elites 均匀随机抽取

 child = crossover(parent_1, parent_2)

 以 35% 概率:

 child = mutate(child)


```
    如果 child 深度超过 3:
        child = 重新随机生成表达式

    next_population 加入 child

    随机生成 12 个新表达式作为移民
    population = next_population, 数量恢复为 80

candidates = hall 中适应度最高的前 600 个
selected = []

依次检查 candidates:
    如果该峰值月份已保留 3 个: 跳过
    如果与任一 selected 因子的绝对相关 >= 0.80: 跳过
    否则加入 selected
    selected 达到 20 个时停止

保存 factor_library.json 和 factor_library.csv
在全部 2026 年前历史上重新计算验证统计, 但不改变 selected
```

11. 当前实现的透明限制

1. **tournament_size=5 当前没有生效。** 配置文件中虽然保留了这个参数, 但代码没有执行锦标赛选择。父本实际是从前 12 名精英中均匀随机抽取。
2. **没有公式等价化简。** 交换加法顺序、双重取反、对非负排名取绝对值等冗余结构仍可能作为不同个体存在。
3. **遗传树没有时间序列算子。** 它不能直接生成 `lag(x, 5)`、`rolling_mean(x, 20)` 或趋势斜率; 历史信息只来自 RSI、MACD、BOLL、5/20 日收益和 5 日资金流等预先计算好的终端特征。
4. **搜索月份和股票是固定抽样。** 这提高了速度和可复现性, 但适应度仍可能受抽样市场阶段影响。
5. **阶段性目标容易偏向特殊市场状态。** Top-3 绝对月 IC 的设计符合“至少某个月有效”的目标, 但不能证明因子长期稳定。
6. **峰值月份限制依据搜索样本。** 全量历史复核后峰值月份可能变化, 而且当前实现不会据此重新执行月份配额筛选。
7. **相关性去重是贪心的。** 先出现的高适应度因子会占据位置, 后续候选即使具有其他价值, 只要与它相关达到阈值就会被跳过。

这些限制不代表当前结果无效, 但在解释因子库和设计下一版遗传搜索时需要明确考虑。

使用到的基础特征

所有名称以 `_rank_cs` 结尾的字段，都是该字段在同一交易日全部股票中的百分位排名，通常位于 `[0, 1]`。

```
turnover_rate      = 成交量 / 流通股本，对应普通换手率
turnover_rate_f    = 成交量 / 自由流通股本，对应自由流通换手率
turnover_rank_cs   = turnover_rate_f 的当日横截面排名
amount_rank_cs     = 成交额 amount 的当日横截面排名
vol_rank_cs        = 成交量 vol 的当日横截面排名
liquidity_rank_cs  = circ_mv * turnover_rate_f 的当日横截面排名
total_mv_rank_cs   = 总市值的当日横截面排名
circ_mv_rank_cs    = 流通市值的当日横截面排名
dv_ttm_rank_cs     = 近 12 月股息率的当日横截面排名

ret_1d_rank_cs     = 1 日前复权收益的当日横截面排名
ret_5d             = close_qfq(t) / close_qfq(t-5) - 1
ret_5d_rank_cs     = ret_5d 的当日横截面排名
ret_20d            = close_qfq(t) / close_qfq(t-20) - 1
ret_20d_rank_cs    = ret_20d 的当日横截面排名
intraday_ret       = close_qfq / open_qfq - 1
lower_shadow        = min(open_qfq, close_qfq) / low_qfq - 1
range_rank_cs      = (high_qfq / low_qfq - 1) 的当日横截面排名

macd_dif           = MACD 快慢线之差
macd_dea           = DIF 的平滑信号线
macd               = MACD 柱值
rsi_6_rank_cs      = 6 日 RSI 的当日横截面排名
rsi_24             = 24 日 RSI 原始值
cci                = CCI 顺势指标原始值
boll_width         = (boll_upper - boll_lower) / boll_mid
boll_position       = (close_qfq - boll_lower) / (boll_upper - boll_lower)
volume_ratio        = 当日量比原始值

sm_net_amount_ratio = (小单买入额 - 小单卖出额) / 当日成交额
md_net_amount_ratio = (中单买入额 - 中单卖出额) / 当日成交额
lg_net_amount_ratio = (大单买入额 - 大单卖出额) / 当日成交额
lg_buy_amount_ratio = 大单买入额 / 当日成交额
lg_net_amount_ratio_5d = lg_net_amount_ratio 最近 5 日滚动和
net_mf_rank_cs      = 总净流入额占成交额比例的当日横截面排名
```

资金流金额单位在计算前已统一：`daily.amount` 从千元除以 10 转成万元，再与 `moneyflow` 的万元口径相除。

20 个正式因子

表中的峰值 IC 来自全部 2026 年前历史数据。它是该因子绝对月均 IC 最大月份的有符号 IC；负值表示当月公式值越高，未来 5 日收益排名反而越低。

factor_01

```
turnover_rate_f * (macd_dea + signed_sqrt(turnover_rank_cs))
```

自由流通换手率乘以“MACD 信号线 + 换手排名压缩值”。偏向换手活跃度和趋势状态的交互。峰值：201607，IC -0.2808。

factor_02

```
turnover_rate_f * signed_sqrt(boll_width)
```

自由流通换手率与布林带宽度的交互，描述高换手和高波动是否同时出现。峰值：201607，IC -0.2731。

factor_03

```
turnover_rate_f * (macd_dea + total_mv_rank_cs)
```

换手率与“趋势信号 + 市值位置”的交互。峰值：201607，IC -0.2554。

factor_04

```
abs(amount_rank_cs)
```

由于成交额排名本身非负，该公式实际等价于 `amount_rank_cs`，即当日成交额横截面排名。`abs` 是遗传搜索留下的冗余算子。峰值：201505，IC -0.2555。

factor_05

```
abs(min(range_rank_cs / lower_shadow, circ_mv_rank_cs))
```

先用下影线幅度缩放日内振幅排名，再与流通市值排名取较小值，最后取绝对值。下影线接近零时该表达式可能缺失。峰值：202401，IC +0.2974。

factor_06

```
min(  
    amount_rank_cs,  
    signed_log(ret_20d_rank_cs - lg_net_amount_ratio_5d)  
)
```

在成交额排名与“20 日动量排名减去大单 5 日净流入比例”之间取较小值，试图描述成交活跃、趋势和大单资金背离。峰值：202504，IC -0.2127。

factor_07

```
turnover_rate_f * (-rsi_6_rank_cs)
```

自由流通换手率乘以负的短期 RSI 排名，是高换手与短期超买/超卖的反向交互。峰值：201904，IC +0.2138。

factor_08

```
signed_sqrt(range_rank_cs)
```

对振幅横截面排名开平方。因为排名非负且开平方单调，该因子的排序与 range_rank_cs 完全相同，只改变数值间距。峰值：202401，IC -0.2449。

factor_09

```
turnover_rate /  
max(lg_buy_amount_ratio, max(circ_mv_rank_cs, liquidity_rank_cs))
```

普通换手率除以“大单买入占比、流通市值排名、流动性排名”三者中的最大值。峰值：202401，IC -0.3231。

factor_10

```
abs(min(
    range_rank_cs / lower_shadow,
    signed_sqrt(vol_rank_cs)
))
```

下影线缩放后的振幅排名，与成交量排名平方根取较小值。峰值： 201501，IC -0.2485。

factor_11

```
(
    min(dv_ttm_rank_cs, lower_shadow)
    + min(ret_5d_rank_cs, volume_ratio)
)
*
max(ret_20d, amount_rank_cs, lg_net_amount_ratio)
```

前半部分融合股息率、下影线、5日动量和量比；后半部分取20日收益、成交额排名和大单净流入比例中的最大状态，再做乘法交互。峰值： 202411，IC -0.2358。

factor_12

```
max(
    ret_5d_rank_cs - md_net_amount_ratio
    + min(turnover_rate_f, total_mv_rank_cs),
    macd_dea
)
```

比较“5日动量与中单资金差异，加上换手/市值较小值”和MACD信号线，取较大者。峰值： 202401，IC +0.2402。

factor_13

```
-min(signed_sqrt(total_mv_rank_cs), macd_dea)
```

市值排名平方根与MACD信号线取较小值后反号，偏向规模和趋势的下限状态。峰值： 201802，IC +0.2445。

factor_14

```
max(macd, boll_position)
* (macd_dea + signed_sqrt(turnover_rank_cs))
```

MACD 柱值与布林带位置取较大值，再乘以趋势信号和换手排名组合。峰值：201504，IC -0.2013。

factor_15

```
max(
    min(abs(volume_ratio), ret_1d_rank_cs * intraday_ret),
    signed_sqrt(max(ret_20d_rank_cs, total_mv_rank_cs))
)
```

比较两种状态：短线量比与日内收益交互，以及20日动量/市值较大值的平方根，最终取较大者。峰值：201603，IC -0.2199。

factor_16

```
-boll_width
```

布林带宽度直接取反。公式值越高代表布林带越窄。峰值：202401，IC +0.2656。

factor_17

```
min(
    min(liquidity_rank_cs + net_mf_rank_cs, abs(ret_5d)),
    intraday_ret - circ_mv_rank_cs - rsi_6_rank_cs
)
```

在“流动性与资金净流入、5日绝对收益的较小值”和“日内收益减规模排名及短期RSI排名”之间再取较小值。峰值：202401，IC -0.2622。

factor_18

```
range_rank_cs * ret_20d_rank_cs
```

振幅排名与20日动量排名的乘积，高值表示高波动和强动量同时存在。峰值：201904，IC -0.1846。

factor_19

```
signed_sqrt(turnover_rank_cs)
/ (sm_net_amount_ratio + circ_mv_rank_cs)
/ rsi_24
```

换手排名平方根，依次除以“小单净流入比例 + 流通市值排名”和24日RSI。分母接近零时可能缺失或产生较大值，随后会被裁剪。峰值：202401，IC -0.2899。

factor_20

```
signed_sqrt((total_mv_rank_cs - turnover_rate_f) * cci)
```

总市值排名减去自由流通换手率，再与CCI相乘并做保号平方根压缩。峰值：202401，IC -0.1825。

如何理解这些方向

这些因子是按“历史至少部分月份高绝对IC”保留的，并没有要求公式方向长期固定。因此不能把factor_01简单写成“越高越好”或“越低越好”：

- IC投票策略每5日根据近期历史重新判断方向。
- LightGBM回归版自行学习各因子的方向和交互。
- LambdaRank版直接学习哪些组合更可能进入未来收益顶部。

目前不少因子的历史峰值IC为负，这不是计算错误，而是说明这些公式在相应月份主要作为反向因子使用。